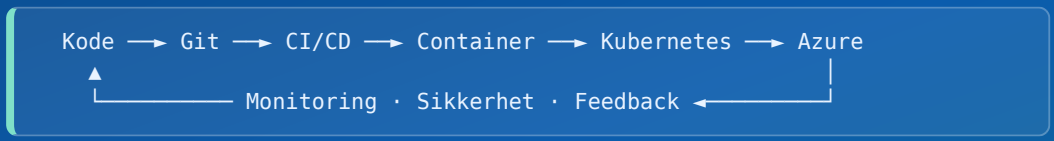
LÆREBOK · NOROFF DEVOPS ENGINEERING

DEVOPS, CLOUD & MICRO-SOFT AZURE

Den komplette guiden fra nybegynner til profesjonell



IT-grunnlag · Nettverk · Linux · Python · Git · Cloud · Azure (AZ-900/AZ-104)
DevOps · Docker · Kubernetes · Terraform · CI/CD · Sikkerhet · MLOps
12 praktiske prosjekter · Quiz med fasit · Ordliste · 12-måneders læringsplan

Utgave oktober 2026 · Skrevet på norsk med engelske fagbegreper

Innholdsfortegnelse

Introduksjon

- Velkommen
- Hvordan bruke boken
- En ærlig forventningsavklaring
- Om kostnader og oppdatert informasjon
- Læringsreisen – oversikt
- Produksjonsplan og kapitteloversikt

Del 1 – Forstå IT fra bunnen

- Hva er IT?
- Hvordan fungerer en datamaskin?
- Hva er et operativsystem?
- Hva er en server?
- Hva er en applikasjon?
- Hva er en database?
- Hvordan fungerer internett?
- Hva er en IP-adresse?
- Hva er DNS?
- Hva er HTTP og HTTPS?
- Hva er et API?
- Hva skjer når du åpner en nettside?
- Klient–server-arkitektur
- Hva er virtualisering?
- Hvordan alt henger sammen
- Kapitteloppsummering
- Øvingsoppgaver
- Quiz

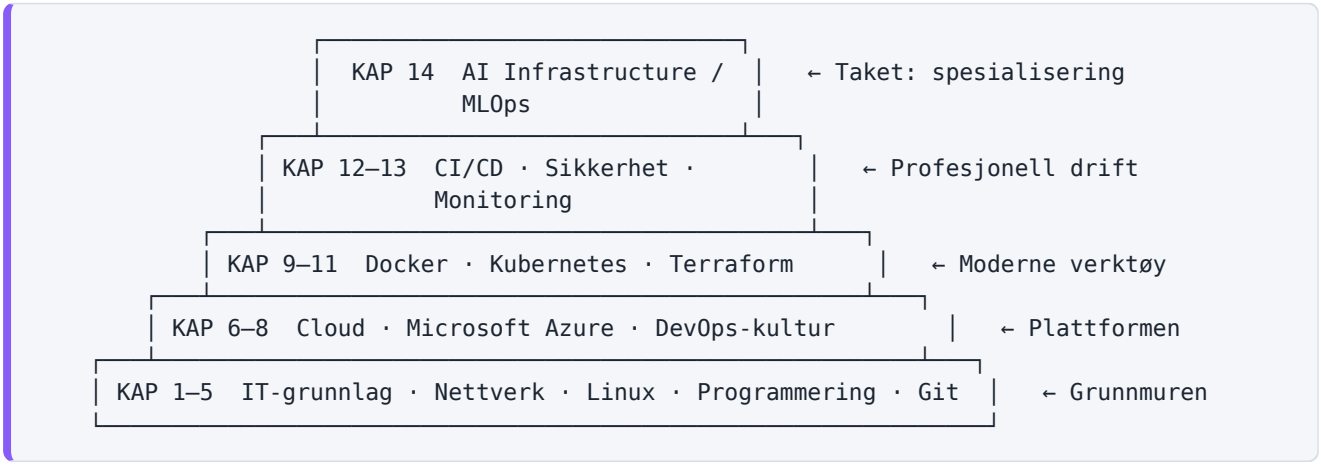
Introduksjon

Velkommen

Denne boken er skrevet for deg som studerer DevOps Engineering, er ganske ny innen IT, og ønsker å forstå **hvorfor** ting fungerer – ikke bare pugge ord.

Mange lærebøker starter med «Kubernetes er en orkestreringsplattform for containere» og forventer at du skal nikke. Denne boken gjør det motsatte. Vi starter med hva en datamaskin faktisk er, bygger videre med nettverk, Linux og programmering, og først når grunnmuren står, bygger vi Cloud, Azure, Docker, Kubernetes, Terraform, CI/CD, sikkerhet og til slutt AI-infrastruktur oppå.

Tenk på boken som et hus:



Hvis du hopper rett til taket, faller huset sammen. Hvis du bygger grunnmuren skikkelig, blir alt det andre mye enklere.

Hvordan bruke boken

Læringsmetoden i hvert kapittel

Hvert viktig konsept forklares med den samme faste oppskriften, slik at du alltid vet hvor du finner hva:

Steg	Spørsmål	Hva du får
1	**Hva er det?**	Forklaring med helt enkle ord
2	**Hvorfor finnes det?**	Problemet teknologien løser
3	**Hvordan fungerer det?**	Steg-for-steg teknisk forklaring
4	**Hverdagsanalogi**	Restaurant, lager, bygning, bil ...

Steg	Spørsmål	Hva du får
5	**Fra arbeidslivet**	Hvordan en ekte bedrift bruker det
6	**Diagram**	Visuell forklaring
7	**Praktisk**	Kode, kommandoer, oppsett
8	**Vanlige feil**	Det nybegynnere misforstår
9	**Husk**	Kort oppsummering
10	**Test deg selv**	Kontrollspørsmål med forklarte svar

For små begreper slår vi noen av stegene sammen, slik at boken ikke blir kunstig lang. For de store temaene (for eksempel Docker, Kubernetes og Azure-tjenestene) får du hele oppskriften.

Faste bokser i teksten

> **Hverdagsanalogi:** Sammenligner teknologien med noe du kjenner fra før.

> **Fra arbeidslivet:** Viser hvordan dette brukes i en ekte bedrift.

> **Vanlig feil:** Noe nybegynnere (og erfarne!) ofte gjør feil.

> **Husk:** Det viktigste du må ta med deg.

> **Kostnad:** Advarsel om ting som kan koste penger i Cloud.

> **Fordypning:** Avansert stoff. Du kan hoppe over dette første gang du leser.

Kodeblokker og terminal

Kommandoer du skal skrive i terminalen vises slik. Tegnet `$` betyr «skriv dette som vanlig bruker» og skal **ikke** skrives inn. Linjer uten `$` er resultatet du får tilbake:

```
$ whoami
student
```

Kommentarer i kode starter med `#` og forklarer hva linjen gjør. De er der for deg – maskinen ignorerer dem.

Engelske begreper

IT-bransjen snakker engelsk, også i Norge. Første gang et begrep dukker opp, får du både det norske og det engelske ordet, for eksempel «lastbalansering (*load balancing*)». Senere bruker vi ofte det engelske ordet, fordi det er det du vil høre på jobb. Bakerst i boken finner du en ordliste.

Tre måter å lese boken på

1. **Fra start til slutt** – anbefalt hvis du er helt ny. Kapitlene bygger på hverandre.
2. **Som oppslagsverk** – bruk innholdsfortegnelsen, ordlisten og hurtigreferansene.
3. **Prosjektdrevet** – start med et prosjekt i kapittel 15, og les kapitlene prosjektet viser til når du står fast.

> **Husk:** Du lærer DevOps med fingrene, ikke bare med øynene. Skriv kommandoene selv. Gjør feil. Fiks dem. Det er slik ferdighetene faktisk bygges.

En ærlig forventningsavklaring

Å lese denne boken gjør deg ikke til senioringeniør. Ingen bok kan det. Det boken *kan* gjøre, er å gi deg:

- et solid mentalt kart over hvordan alle bitene henger sammen,
- presise forklaringer du kan komme tilbake til,
- oppgaver og prosjekter som bygger ekte, praktiske ferdigheter,
- en realistisk plan for de neste tolv månedene.

Erfaring kommer av å bygge, feile, feilsøke og bygge igjen – helst sammen med andre. En god juniorkandidat er ikke en som kan alt, men en som forstår grunnprinsippene, kan lære raskt og kan vise frem ekte prosjekter.

Om kostnader og oppdatert informasjon

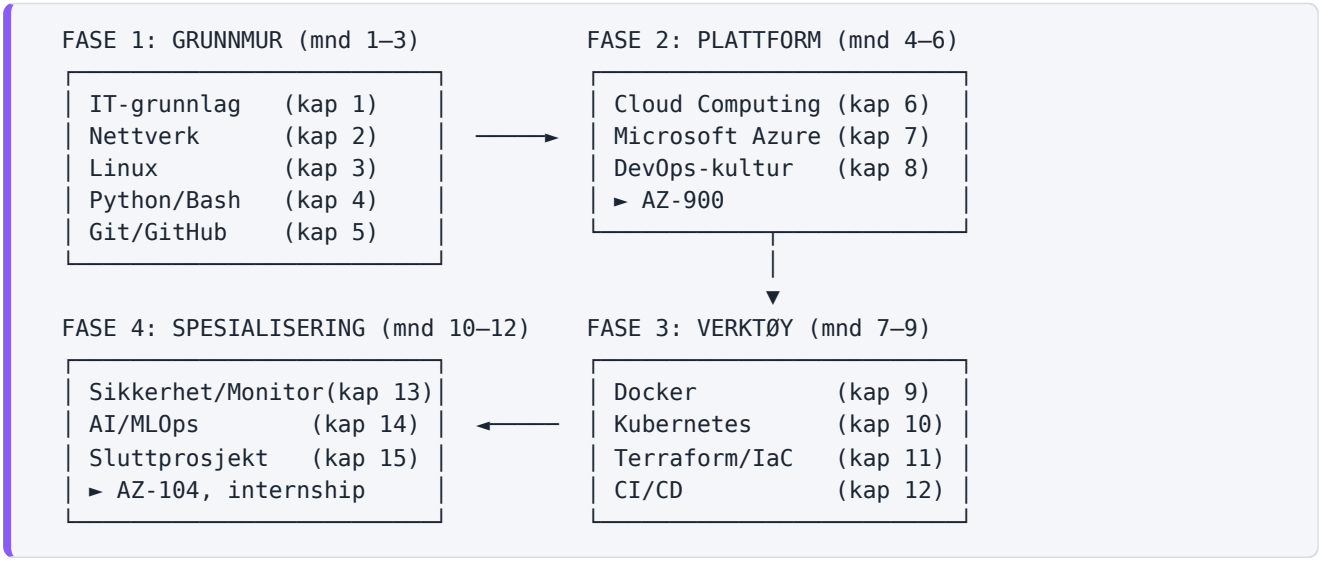
> **Kostnad:** Cloud-tjenester koster ekte penger. Gratiskontoer og studentkreditter har grenser. Hvert prosjekt i boken har en egen seksjon om **opprydding**. Følg den hver gang. Sett opp et budsjettvarsel (*budget alert*) i Azure før du gjør noe annet – det viser vi i kapittel 7.

Teknologi endrer seg raskt. Tjenester får nye navn (Azure Active Directory heter nå *Microsoft Entra ID*), priser endres, og verktøy får nye versjoner. Informasjonen i boken er skrevet ut fra offentlig dokumentasjon fra Microsoft Learn, Docker, Kubernetes, HashiCorp, GitHub og Python slik den var kjent per **oktober 2026**. Der noe er spesielt utsatt for endring, står det en merknad. **Sjekk alltid of-**

fisiell dokumentasjon før du stoler på priser, kvoter, eksamensinnhold og versjonsnumre. En samlet kildeliste med lenker finner du i vedlegg D.

Boken bruker tekstbaserte diagrammer (laget med tegn) i stedet for skjermbilder. Det er et bevisst valg: skjermbilder av Azure Portal blir raskt utdaterte fordi Microsoft endrer utseendet ofte, mens diagrammene forklarer prinsippene som ikke endrer seg. Der et skjermbilde ville vært nyttig, beskriver teksten hva du skal se etter i portalen.

Læringsreisen – oversikt



Produksjonsplan og kapitteloversikt

Boken er bygget opp i 15 deler pluss vedlegg. Her er planen for hele boken, med hva hvert kapittel inneholder og hvilke praktiske leveranser du sitter igjen med.

Del	Kapittel	Innhold	Du kan etterpå
1	Forstå IT fra bunnen	Datamaskin, OS, server, database, applikasjon, frontend/backend, internett, IP, DNS, HTTP(S), API, klient–server, virtualisering	Forklare hva som skjer når du åpner en nettside

Del	Kapittel	Innhold	Du kan etterpå
2	Nettverk for DevOps	IPv4/IPv6, subnetting/CIDR, DNS, DHCP, TCP/UDP, porter, ruter/svitsj, brannmur, VPN, NAT, lastbalansering, proxy, TLS, statuskoder, SSH, feilsøking	Planlegge et VNet og feilsøke tilkoblingsproblemer
3	Linux	Distribusjoner, shell, filsystem, brukere, rettigheter, prosesser, systemd, pakker, SSH, Bash, cron, miljøvariabler, logger, sikkerhet	Drifte en Linux-server fra terminalen
4	Programmering for DevOps	Python fra null, datatyper, løkker, funksjoner, JSON/YAML, API-kall, feilhåndtering, logging, Bash, automatisering, Azure SDK	Skrive egne automatiseringsskript
5	Git og GitHub	Repo, commit, branch, merge, PR, konflikter, arbeidsflyter, GitHub Actions, samarbeid	Jobbe profesjonelt med versjonskontroll
6	Cloud Computing	Datasentre, IaaS/PaaS/SaaS, regioner/soner, skalering, HA, DR, sikkerhet, prising, delt ansvar, AWS/Azure/GCP	Velge riktig skytjeneste for et behov
7	Microsoft Azure A–Å	30+ tjenester med opprettelse, sikkerhet, kostnad og øvelser; AZ-900 og AZ-104 med prøveeksamener	Administrere grunnleggende Azure-miljøer
8	DevOps Engineering	Kultur, Agile/Scrum, SDLC, CI/CD, deploystrategier, observability, hendelser, SRE, Platform Engineering, DevSecOps	Forklare DevOps i et intervju

Del	Kapittel	Innhold	Du kan etterpå
9	Docker	Images, containere, Dockerfile, Compose, registry, volumes, nettverk, sikkerhet, feilsøking	Containerisere en applikasjon
10	Kubernetes	Cluster, noder, pods, deployments, services, ingress, config, secrets, lagring, autoskalering, Helm, AKS	Deploye en app på Kubernetes
11	Infrastructure as Code	Terraform, state, moduler, remote state, Bicep, Ansible, GitOps	Bygge Azure-infrastruktur med kode
12	CI/CD og automatisering	GitHub Actions, Azure Pipelines, Jenkins, testing, artefakter, hemmeligheter, rollback, blue-green, canary	Bygge en pipeline fra kode til produksjon
13	Sikkerhet og overvåking	IAM, minste privilegium, kryptering, Zero Trust, skanning, backup, logger, metrikker, tracing, Prometheus, Grafana, hendelseshåndtering	Sikre og overvåke et system
14	AI Infrastructure og MLOps	GPU, modellservering, ML-pipelines, Azure ML, Azure AI-tjenester, LLM-infrastruktur, kostnader, karrierevei	Forstå hvordan AI kjøres i produksjon
15	Virkelige prosjekter	12 prosjekter med økende vanskelighetsgrad	Vise frem en sterk portefølje
A–E	Vedlegg	Ordliste, hurtigreferanser, sertifiseringer, kilder, 12-måneders læringsplan	Repetere og planlegge videre

Slik er hvert kapittel bygget opp: læringsmål → forklaringer med læringsmetoden → praktiske øvelser → kapitteloppsummering → quiz med fasit.

Kvalitetsprinsipper for boken:

- Ingen plassholdere og ingen «forklaring kommer senere».
- Alle kommandoer er ekte kommandoer fra offisielle verktøy.
- Grunnleggende stoff og fordypning holdes adskilt.
- Kostnader og gratisalternativer nevnes eksplisitt.
- Informasjon som endres ofte er merket med dato og kilde.

Del 1 – Forstå IT fra bunnen

Læringsmål for kapitlet. Når du er ferdig, skal du kunne:

- forklare hva CPU, RAM, lagring og operativsystem gjør, og hvordan de samarbeider,
- forklare forskjellen på en server og en vanlig PC,
- forklare hva en applikasjon, en database, frontend og backend er,
- beskrive steg for steg hva som skjer når du skriver `www.vg.no` i nettleseren,
- forklare IP-adresser, DNS, HTTP/HTTPS og API på et grunnleggende nivå,
- forklare klient–server-arkitektur og virtualisering.

Hva er IT?

Hva er det? IT (informasjonsteknologi, *information technology*) er all teknologi vi bruker for å **lagre, behandle og sende informasjon**. Det inkluderer maskinvare (*hardware*) som datamaskiner, servere og nettverksutstyr, og programvare (*software*) som operativsystemer, apper og databaser.

Hvorfor finnes det? Bedrifter og samfunnet har enorme mengder informasjon: kundedata, bestillinger, pasientjournaler, lønnsslipper. Uten IT måtte alt dette ligget i papirarkiver og blitt behandlet manuelt.

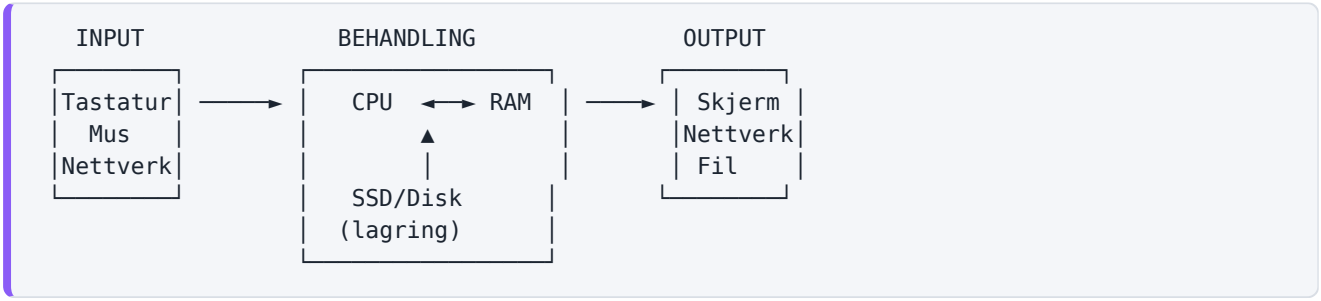
Rollene i IT. IT er et stort felt. Noen vanlige roller:

Rolle	Hva de gjør
Utvikler (*developer*)	Skriver koden som blir til applikasjoner
Driftstekniker (*operations / sysadmin*)	Holder servere og systemer i gang
Nettverksingeniør	Designer og drifter nettverk
DevOps Engineer	Bygger broen mellom utvikling og drift med automatisering
Cloud Engineer	Bygger og drifter løsninger i skyen (Azure, AWS, GCP)
Site Reliability Engineer (SRE)	Sørger for at systemer er pålitelige, ofte med kode
Sikkerhetsingeniør	Beskytter systemer mot angrep
MLOps / AI Infrastructure Engineer	Bygger plattformer som trener og kjører AI-modeller

> **Husk:** En DevOps Engineer må forstå litt av *alt* – maskinvare, nettverk, operativsystem, kode og skyen – fordi jobben er å få alle delene til å spille sammen.

Hvordan fungerer en datamaskin?

Hva er det? En datamaskin er en maskin som tar imot data (*input*), behandler dem (*processing*), lagrer dem (*storage*) og gir et resultat (*output*).



CPU – hjernen

Hva er det? CPU (*Central Processing Unit*, prosessor) utfører instruksjonene i programmene. Den regner, sammenligner og flytter data.

Hvordan fungerer det? CPU-en henter en instruksjon, tolker den og utfører den – milliarder av ganger i sekundet. Hastigheten måles i gigahertz (GHz). Moderne CPU-er har flere **kjerner** (*cores*), og hver kjerne kan jobbe med sin egen oppgave samtidig.

> **Hverdagsanalogi:** CPU-en er kokken på en restaurant. Flere kjerner er flere kokker. Fire kokker kan lage fire retter samtidig, men én enkelt rett blir ikke nødvendigvis raskere av at det står fire kokker rundt den.

> **Fra arbeidslivet:** Når du velger størrelse på en virtuell maskin i Azure, velger du antall **vCPU** (virtuelle CPU-kjerner). En liten webserver trenger kanskje 1–2 vCPU, mens en database for en nettbutikk kan trenge 8–16.

RAM – arbeidsbenken

Hva er det? RAM (*Random Access Memory*, arbeidsminne) er der datamaskinen holder det den jobber med *akkurat nå*. RAM er ekstremt rask, men **flyktig** (*volatile*): alt forsvinner når strømmen går.

> **Hverdagsanalogi:** RAM er kokkens benk. Ingrediensene kokken bruker akkurat nå ligger på benken. Er benken for liten, må kokken gå til lageret hele tiden, og alt går tregt.

> **Fra arbeidslivet:** Når en server går tom for RAM, begynner operativsystemet å flytte data til disk (*swapping*) – det gjør alt mye tregere. På Linux kan prosessen til og med bli drept av «OOM killer» (*Out Of Memory*). I Kubernetes ser du dette som statusen `OOMKilled`.

Lagring – SSD og harddisk

Hva er det? Lagring (*storage*) beholder data selv når maskinen er slått av: filer, programmer, operativsystemet.

Type	Hvordan	Fart	Typisk bruk
HDD (harddisk)	Roterende magnetiske plater	Treg	Billig masselagring, arkiv
SSD	Flash-minne, ingen bevegelige deler	Rask	Operativsystem, apper
NVMe SSD	SSD koblet direkte på PCIe-bussen	Svært rask	Databaser, krevende systemer

> **Hverdagsanalogi:** Lagring er restaurantens kjølerom. Alt ligger trygt der over natten, men det tar lengre tid å hente noe derfra enn fra benken (RAM).

Hastighetshierarkiet er viktig å forstå:

```
Raskest, minst, dyrest
  ▲   CPU-cache    (nanosekunder)
  │   RAM          (~100 nanosekunder)
  │   NVMe SSD     (mikrosekunder)
  │   HDD          (millisekunder)
  │   Nettverk/sky-lagring (millisekunder og mer)
  ▼
Tregest, størst, billigst
```

Hovedkort, nettverkskort og GPU

- **Hovedkort** (*motherboard*) kobler alle delene sammen.
- **Nettverkskort** (*NIC – Network Interface Card*) kobler maskinen til nettverket.
- **GPU** (*Graphics Processing Unit*) har tusenvis av små kjerner som er gode på å gjøre samme regneoperasjon på mye data samtidig. Opprinnelig laget for grafikk – i dag helt sentral for AI (se kapittel 14).

Hva er et operativsystem?

Hva er det? Operativsystemet (*Operating System, OS*) er programvaren som styrer maskinvaren og lar andre programmer bruke den. Eksempler: Windows, macOS, Linux, Android, iOS.

Hvorfor finnes det? Uten et OS måtte hvert program selv vite hvordan det skal snakke med akkurat din SSD, ditt nettverkskort og din skjerm. OS-et gir et felles grensesnitt.

Hva gjør OS-et?

1. **Prosessstyring** – bestemmer hvilket program som får bruke CPU-en når.
2. **Minnehåndtering** – deler ut RAM til programmene og hindrer dem i å lese hverandres minne.
3. **Filsystem** – organiserer data på disken i filer og mapper.
4. **Drivere** – snakker med maskinvaren.
5. **Sikkerhet** – brukere, passord og rettigheter.
6. **Nettverk** – sender og mottar data.



> **Hverdagsanalogi:** OS-et er restaurantsjefen. Kokkene (programmene) må spørre sjefen om plass på benken (RAM), tid ved komfyren (CPU) og hylleplass i kjølerommet (disk).

Kjernen (*kernel*) er selve hjertet i operativsystemet. Linux er egentlig navnet på en kjerne; rundt den bygges hele operativsystemer som Ubuntu og Red Hat (mer i kapittel 3).

Hva er en server?

Hva er det? En server er en datamaskin (eller et program) som **leverer tjenester til andre maskiner** over et nettverk. Ordet kommer fra engelsk *serve* – å servere.

Hvorfor finnes det? Når du åpner Instagram, ligger ikke bildene på telefonen din. De ligger på servere som leverer dem til deg og millioner av andre.

Server vs. vanlig PC

Egenskap	Vanlig PC	Server
Formål	Én bruker, interaktivt arbeid	Mange brukere, leverer tjenester
Oppetid	Slås av om kvelden	Går 24/7 i årevis
Skjerm/tastatur	Ja	Vanligvis ikke (*headless*), styres over nettverk
Operativsystem	Windows/macOS	Oftest Linux, også Windows Server
Maskinvare	Forbrukerdeler	Redundante strømforsyninger, ECC-RAM, RAID-disker
Plassering	Kontor/hjemme	Serverrom eller datasenter, i rack
Grafisk grensesnitt	Ja	Ofte bare kommandolinje

> **Vanlig feil:** Å tro at en server må være en spesiell type maskin. Teknisk sett kan din egen laptop være en server – det handler om *rollen*. Men i produksjon bruker man maskinvare som er laget for å tåle kontinuerlig drift.

> **Fra arbeidslivet:** I dag kjører de fleste «servere» som **virtuelle maskiner** eller **containere** i et datasenter eid av en skyleverandør. Du ser aldri den fysiske maskinen.

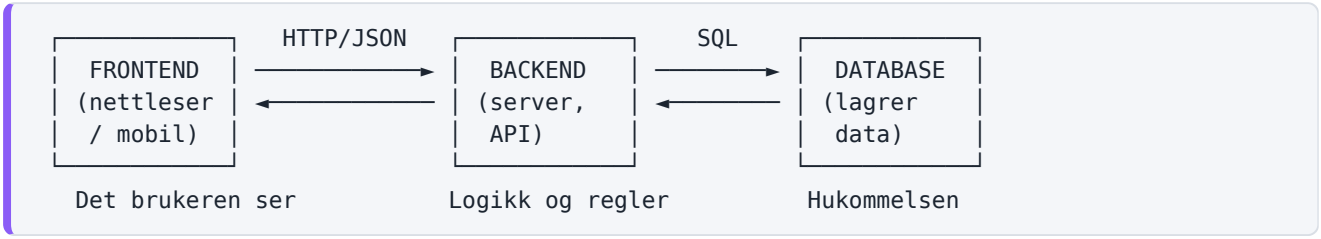
Vanlige typer servere (roller):

- **Webserver** – leverer nettsider (nginx, Apache, IIS).
- **Applikasjonsserver** – kjører forretningslogikken (Python, Java, Node.js).
- **Databaseserver** – lagrer strukturerte data (PostgreSQL, SQL Server, MySQL).
- **Filserver** – deler filer.
- **DNS-server** – oversetter navn til IP-adresser.
- **E-postserver** – sender og mottar e-post.

Hva er en applikasjon?

Hva er det? En applikasjon (app, program) er programvare som løser en oppgave for en bruker: en nettbank, Spotify, et lønnssystem.

Moderne applikasjoner består som regel av flere deler som snakker sammen:



Frontend og backend

Frontend er det brukeren ser og klikker på: knapper, skjemaer, bilder. Bygges med HTML (struktur), CSS (utseende) og JavaScript (oppførsel), ofte med rammeverk som React, Vue eller Angular. Frontend kjører i brukerens nettleser eller mobil.

Backend er det som skjer bak kulissene: sjekke passord, regne ut priser, hente data, sende e-post. Skrives i språk som Python, Java, C#, Go eller JavaScript (Node.js). Backend kjører på servere.

> **Hverdagsanalogi:** På en restaurant er **frontend** spisesalen og menyen – det gjesten ser. **Backend** er kjøkkenet. **Databasen** er lageret med råvarer. **Servitøren** som tar bestillingen fra salen til kjøkkenet er **API-et**.

> **Fra arbeidslivet:** En DevOps Engineer skriver sjelden frontend-kode, men må vite hvordan både frontend og backend bygges, testes og deployes. Frontend blir ofte statiske filer som legges på en CDN eller Azure Static Web Apps; backend kjøres i containere, App Service eller Kubernetes.

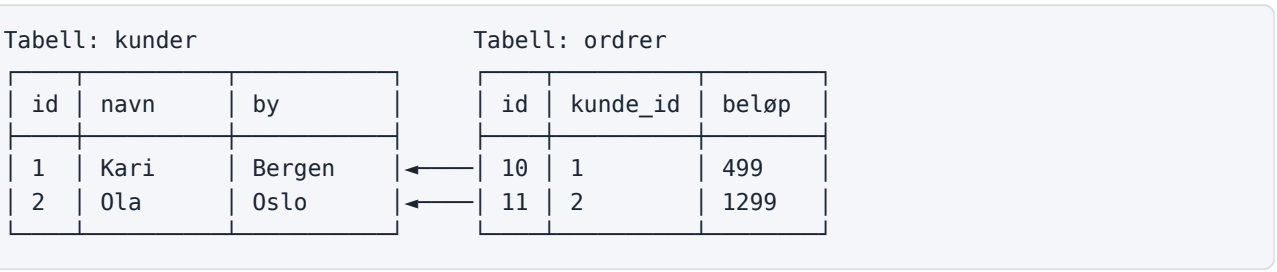
Hva er en database?

Hva er det? En database er et system for å **lagre, organisere og hente data** effektivt og trygt.

Hvorfor finnes det? Du *kunne* lagret alle kundene dine i en tekstfil. Men hva skjer når 10 000 brukere skriver til filen samtidig, når du må finne én kunde blant 50 millioner, eller når strømmen går midt i en betaling? Databaser løser dette.

Relasjonsdatabaser (SQL)

Data lagres i **tabeller** med rader og kolonner – som regneark med strenge regler. Tabeller kobles sammen med **nøkler**.



Man snakker med dem via **SQL** (*Structured Query Language*):

```sql
-- Hent navn og beløp for alle ordrer over 1000 kroner
SELECT kunder.navn, ordrer.beløp
FROM ordrer
JOIN kunder ON kunder.id = ordrer.kunde_id
WHERE ordrer.beløp > 1000;
```

Eksempler: PostgreSQL, MySQL, Microsoft SQL Server, Azure SQL Database.

NoSQL-databaser

Lagrer data i mer fleksible formater: dokumenter (JSON), nøkkel–verdi, grafer eller kolonnefamilier. Eksempler: MongoDB, Redis, Azure Cosmos DB.

	SQL (relasjonell)	NoSQL
Struktur	Fast skjema, tabeller	Fleksibel
Styrke	Konsistens, komplekse spørringer, transaksjoner	Skalering, fart, fleksible data
Typisk bruk	Bank, ordre, regnskap	Sesjoner, cache, IoT, kataloger

> **Husk:** Databaser er nesten alltid den mest kritiske delen av et system. En webserver kan du bygge på nytt på minutter. Mistet data er ofte borte for alltid. Derfor: **backup, backup, backup** – og test at du faktisk kan gjenopprette.

Hvordan fungerer internett?

Hva er det? Internett er et globalt nettverk av nettverk. Millioner av nettverk (hjemmenett, bedriftsnett, mobilnett, datasentre) er koblet sammen av internettleverandører (ISP-er) og sjøkabler, og de bruker felles regler – **protokoller** – for å snakke sammen.

Hvordan fungerer det? Data deles opp i små **pakker** (*packets*). Hver pakke har en avsender-adresse og en mottaker-adresse (IP-adresser). Rutere langs veien leser mottakeradressen og sender pakken videre i riktig retning, litt som postterminaler. Pakkene kan ta forskjellige veier og settes sammen igjen hos mottakeren.

> **Hverdagsanalogi:** Tenk deg at du skal sende en bok til en venn i Tromsø, men posten bare tar postkort. Du river ut sidene, nummererer dem, skriver adressen på hver og sender dem. Noen går via Trondheim, andre via Bodø. Vennen din sorterer dem i riktig rekkefølge og har boken igjen. Det er pakkesvitsjing.

Viktige protokoller du vil møte i hele boken:

Protokoll	Oppgave
IP	Adressering og ruting av pakker
TCP	Pålitelig levering, i riktig rekkefølge
UDP	Rask levering uten garanti
DNS	Oversetter navn til IP-adresser
HTTP/HTTPS	Overfører nettsider og API-data
TLS	Krypterer forbindelsen
SSH	Sikker fjernstyring av servere

Hva er en IP-adresse?

Hva er det? En IP-adresse er en unik adresse for en enhet på et nettverk, omtrent som en gate-adresse. Eksempel (IPv4): `192.168.1.25`.

En IPv4-adresse består av fire tall fra 0 til 255, separert med punktum. Det gir omtrent 4,3 milliarder adresser – og de er nesten brukt opp. Derfor finnes **IPv6**, med adresser som `2001:db8::1`, og et nesten uendelig antall adresser.

Det finnes **private** adresser (brukes inne i hjemmenett og bedriftsnett, f.eks. `192.168.x.x` og `10.x.x.x`) og **offentlige** adresser (synlige på internett). Dette går vi grundig gjennom i kapittel 2.

Hva er DNS?

Hva er det? DNS (*Domain Name System*) er internettets telefonkatalog. Den oversetter navn som `www.noroff.no` til IP-adresser som maskiner forstår.

Hvorfor finnes det? Mennesker husker navn; maskiner bruker tall. Dessuten kan IP-adressen bak et navn endres uten at brukerne merker noe.

```
$ nslookup www.example.com
Name:    www.example.com
Address: 93.184.215.14
```

(IP-adressen du får kan være en annen – den kan endres over tid.)

Vi går dypt inn i DNS i kapittel 2.

Hva er HTTP og HTTPS?

Hva er det? HTTP (*HyperText Transfer Protocol*) er språket nettlesere og webservere bruker for å snakke sammen. HTTPS er HTTP **kryptert med TLS**, slik at ingen underveis kan lese eller endre innholdet.

Hvordan fungerer det? Klienten sender en **forespørsel** (*request*), serveren svarer med en **respons** (*response*).

```
FORESPØRSEL (fra nettleser)          RESPONS (fra server)
GET /index.html HTTP/1.1             HTTP/1.1 200 OK
Host: www.example.com                Content-Type: text/html
User-Agent: Mozilla/5.0 ...          Content-Length: 1256

                                     <html>...</html>
```

En forespørsel har en **metode** som forteller hva klienten vil:

Metode	Betydning	Eksempel
GET	Hent data	Vis en produktside
POST	Opprett noe nytt	Send inn et skjema
PUT / PATCH	Oppdater	Endre profilen din
DELETE	Slett	Slett en kommentar

Responsen har en **statuskode**: `200` (OK), `404` (finnes ikke), `500` (serverfeil). Full liste i kapittel 2.

Du kan selv se HTTP i terminalen med `curl`:

```
$ curl -I https://www.example.com
HTTP/2 200
content-type: text/html
...
```

`-I` betyr «vis bare hodet (*headers*) i responsen».

> **Vanlig feil:** Å tro at HTTPS betyr at nettsiden er trygg. HTTPS betyr bare at *forbindelsen* er kryptert. En svindelside kan fint ha HTTPS.

Hva er et API?

Hva er det? Et API (*Application Programming Interface*) er en avtalt måte for programmer å snakke med hverandre på. Et web-API tar imot HTTP-forespørsler og svarer – oftest med data i **JSON**-format.

Hvorfor finnes det? Et program skal ikke trenge å vite *hvordan* et annet program er bygget inni. Det holder å kjenne «menyen»: hvilke forespørsler du kan sende og hva du får tilbake.

> **Hverdagsanalogi:** API-et er servitøren. Du (klienten) trenger ikke gå inn på kjøkkenet. Du bestiller fra menyen (API-dokumentasjonen), servitøren tar bestillingen til kjøkkenet (serveren) og kommer tilbake med maten (dataene).

Eksempel – hent data fra et offentlig API (GitHub):

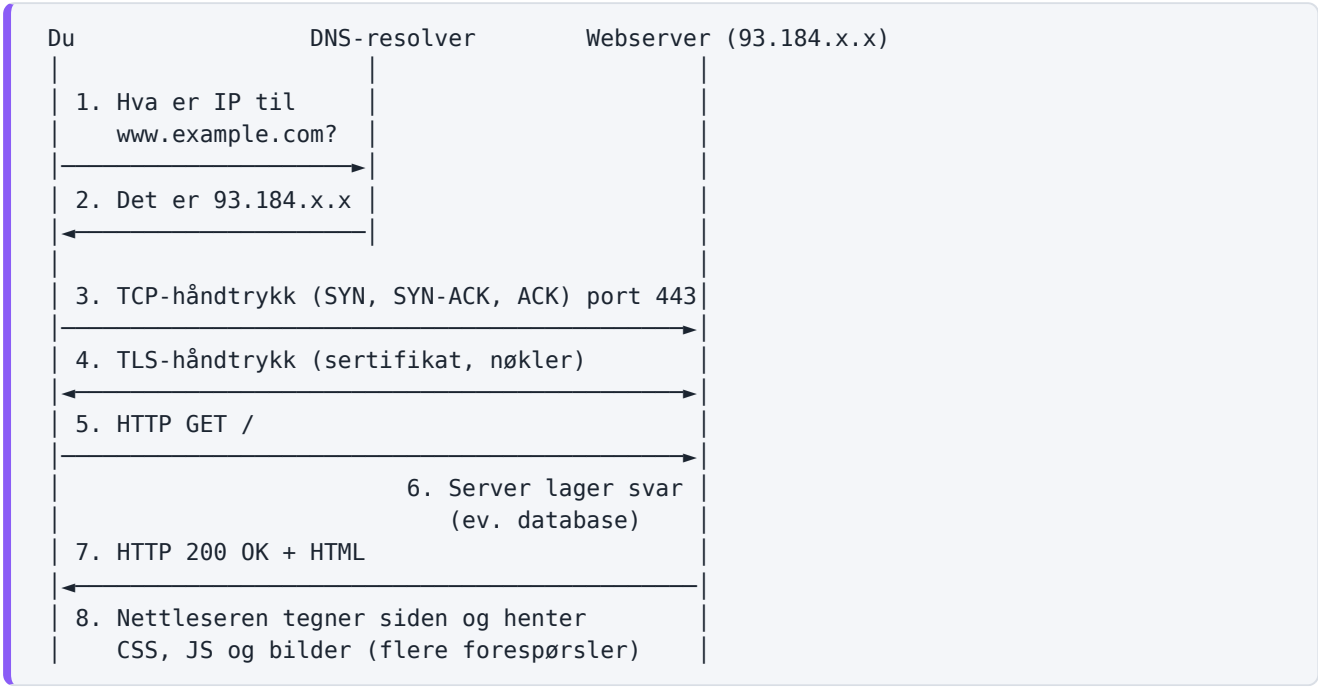
```
$ curl https://api.github.com/users/octocat
{
  "login": "octocat",
  "id": 583231,
  "name": "The Octocat",
  "public_repos": 8,
  ...
}
```

> **Fra arbeidslivet:** Nesten alt i Cloud er API-er. Når du klikker «Create» i Azure Portal, sender portalen egentlig et API-kall til Azure Resource Manager. Azure CLI, Terraform og Bicep sender de samme API-kallene. Dette er grunnen til at alt i skyen *kan* automatiseres.

REST er den vanligste stilen for web-API-er: ressurser har URL-er (`/users/42`), og man bruker HTTP-metodene for å jobbe med dem.

Hva skjer når du åpner en nettside?

Dette er et klassisk intervjuspørsmål. La oss gå gjennom hva som skjer når du skriver `https://www.example.com` og trykker Enter.

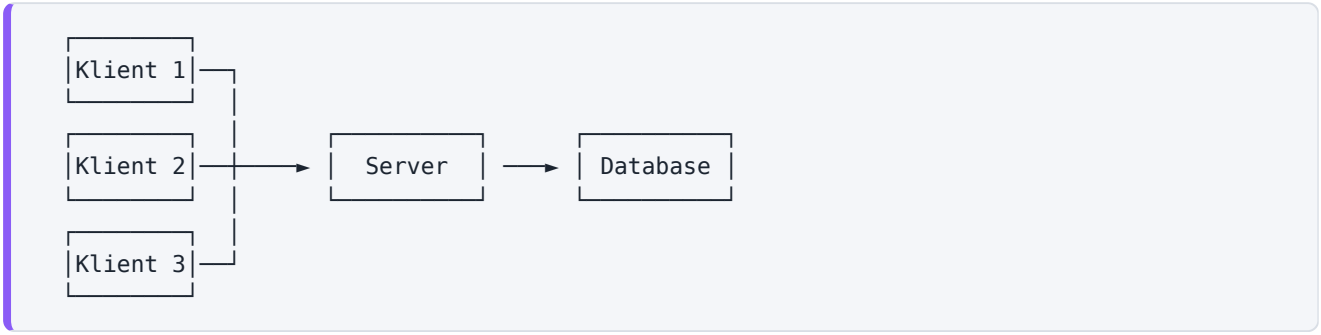


1. **URL tolkes.** Nettleseren ser protokoll (`https`), domenenavn (`www.example.com`) og sti (`/`).
2. **DNS-oppslag.** Nettleseren sjekker sin cache, deretter operativsystemets, deretter spør den en DNS-resolver (ofte hos internettleverandøren eller f.eks. 1.1.1.1). Resolveren finner IP-adressen.
3. **TCP-forbindelse.** Nettleseren åpner en forbindelse til IP-adressen på port 443 (HTTPS) med et tre-veis håndtrykk.
4. **TLS-håndtrykk.** Serveren viser sitt sertifikat; nettleseren sjekker at det er gyldig og utstedt for riktig domene. De blir enige om krypteringsnøkler.
5. **HTTP-forespørsel.** Nettleseren sender `GET /`.
6. **Serveren behandler.** Ofte går forespørselen først gjennom en lastbalanserer, så til en webserver, som kanskje spør en applikasjonsserver og en database.
7. **Respons.** Serveren svarer med statuskode og HTML.
8. **Rendering.** Nettleseren leser HTML, oppdager at den trenger CSS, JavaScript og bilder, og henter dem (ofte fra en CDN). Siden tegnes på skjermen.

> **Husk:** Hvert eneste steg kan feile, og en DevOps Engineer må vite hvordan man feilsøker hvert steg: DNS (`nslookup`), nettverk (`ping`, `traceroute`), port (`nc`, `telnet`), TLS (`openssl s_client`), HTTP (`curl -v`). Dette øver vi på i kapittel 2.

Klient–server-arkitektur

Hva er det? En modell der **klienter** (nettlesere, mobilapper, andre programmer) sender forespørsler, og **servere** svarer. Nesten hele internett er bygget slik.



Fordeler: data og logikk er samlet ett sted (lett å oppdatere og sikre), og klientene kan være enkle.

Utfordring: serveren blir et sentralt punkt. Går den ned, virker ingenting – det er et **single point of failure**. Løsningen er å ha flere servere bak en lastbalanserer (kapittel 2 og 7).

Fra enkelt til avansert finnes det flere arkitekturer:

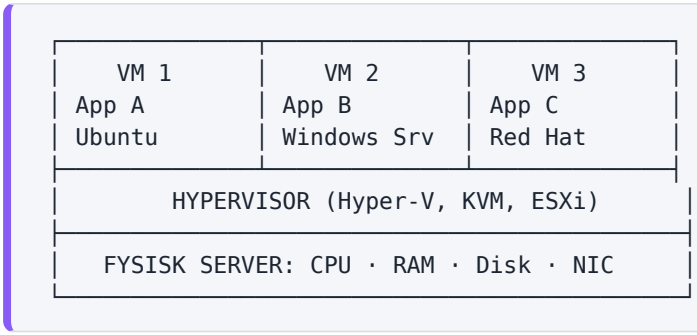
- **Monolitt** – hele applikasjonen er ett stort program. Enkelt å starte med.
- **Tre-lags arkitektur** (*three-tier*) – presentasjon, logikk og data i hvert sitt lag.
- **Mikrotjenester** (*microservices*) – applikasjonen deles i mange små tjenester som snakker via API-er. Fleksibelt, men mer komplekst å drifte. Det er her Docker og Kubernetes virkelig skinner.

Hva er virtualisering?

Hva er det? Virtualisering lar én fysisk maskin late som den er mange separate maskiner – **virtuelle maskiner** (*VM-er*).

Hvorfor finnes det? Før virtualisering kjørte bedrifter typisk én applikasjon per fysisk server (for å unngå at appene ødela for hverandre). De fleste serverne brukte 5–15 % av kapasiteten. Det var sløsing med strøm, plass og penger.

Hvordan fungerer det? En programvare kalt **hypervisor** fordeler den fysiske maskinens CPU, RAM, disk og nettverk mellom flere VM-er. Hver VM har sitt eget operativsystem og tror den er en ekte maskin.



To typer hypervisor:

- **Type 1 (bare metal)** – kjører direkte på maskinvaren. Brukes i datasentre. Eksempler: Microsoft Hyper-V (som Azure bygger på), VMware ESXi, KVM (Linux).

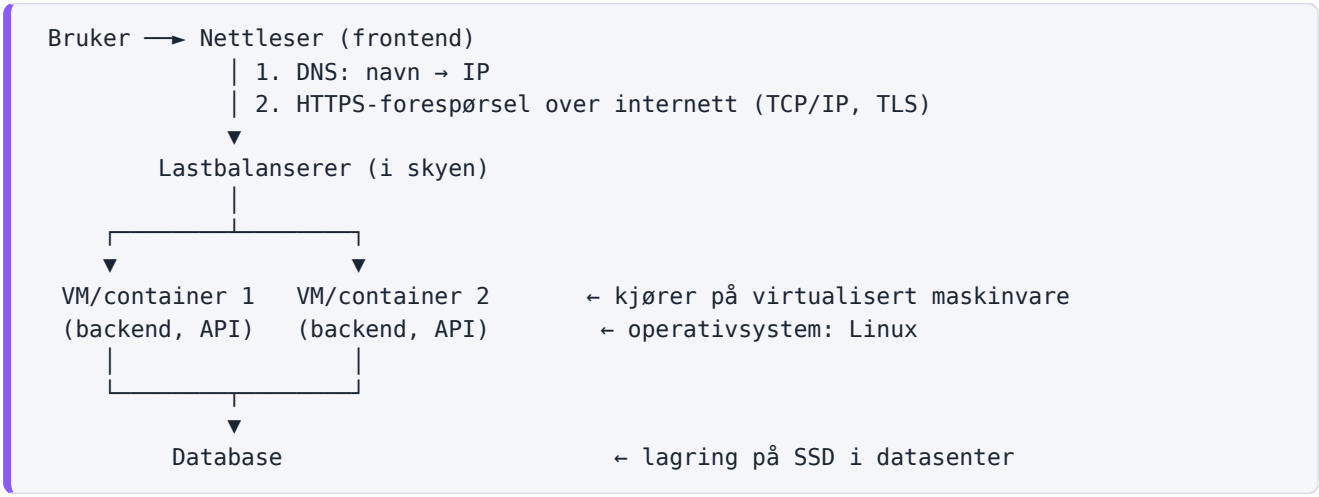
- **Type 2 (hosted)** – kjører som et program oppå et vanlig OS. Brukes på laptoper. Eksempler: VirtualBox, VMware Workstation, Parallels.

> **Hverdagsanalogi:** En fysisk server er en stor bygning. Uten virtualisering leier én bedrift hele bygningen selv om den bare bruker ett rom. Med virtualisering deles bygningen i leiligheter med egne dører og låser. Hypervisoren er vaktmesteren som fordeler strøm og vann.

> **Fra arbeidslivet:** Når du lager en VM i Azure, starter Azure en virtuell maskin på en fysisk server i et av Microsofts datasentre, ved hjelp av en spesialtilpasset Hyper-V. Hele skyen er bygget på virtualisering.

Containere er en lettere form for isolasjon der appene deler operativsystemets kjerne i stedet for å ha hvert sitt OS. Det er temaet i kapittel 9.

Hvordan alt henger sammen



Alt i denne boken handler om å bygge, automatisere, sikre og overvåke denne kjeden.

Kapitteloppsummering

- En datamaskin består av CPU (regner), RAM (raskt korttidsminne), lagring (permanent) og nettverkskort.
- Operativsystemet styrer maskinvaren og lar programmer dele den trygt.
- En server er en rolle: en maskin som leverer tjenester over nettverk, ofte uten skjerm, 24/7.
- En applikasjon består gjerne av frontend (det brukeren ser), backend (logikken) og database (dataene).

- Internett sender data i pakker mellom IP-adresser; DNS oversetter navn til IP; HTTP(S) er språket for web.
- API-er lar programmer snakke sammen; all Cloud-automatisering bygger på API-er.
- Virtualisering lar én fysisk server kjøre mange VM-er – fundamentet for Cloud.

Øvingsoppgaver

1. Åpne Oppgavebehandling (Windows) eller Aktivitetsmonitor (macOS). Finn hvor mange CPU-kjerner, hvor mye RAM og hvor stor disk maskinen din har. Hvilket program bruker mest RAM?
2. Kjør `nslookup www.noroff.no` i terminalen. Hvilken IP-adresse fikk du?
3. Kjør `curl -I https://www.noroff.no`. Hvilken statuskode fikk du? Hvilke *headers* ser du?
4. Tegn (på papir) hva som skjer når du åpner en nettside, med alle åtte stegene. Forklar det høyt for en venn.
5. Last ned VirtualBox og installer Ubuntu Server i en VM. (Dette er grunnlaget for kapittel 3.)

Quiz

1. Hva er forskjellen på RAM og SSD?
2. Hvorfor kjører de fleste servere uten skjerm?
3. Hva er forskjellen på frontend og backend?
4. Hva gjør DNS?
5. Hva betyr statuskode 404?
6. Hva er forskjellen på HTTP og HTTPS?
7. Hva er et API, forklart med en analogi?
8. Hva er en hypervisor?
9. Hva er et *single point of failure*?
10. Hvorfor kan alt i skyen automatiseres?

Fasit

1. **RAM** er raskt, flyktig arbeidsminne som tømmes når strømmen går. **SSD** er permanent lagring som beholder data, men er tregere enn RAM.
2. Fordi de styres over nettverket (f.eks. via SSH), står i datasentre uten fysisk tilgang, og skjerm/grafikk ville bare brukt ressurser.
3. **Frontend** er det brukeren ser og interagerer med (kjører i nettleser/mobil). **Backend** er logikken og dataene bak (kjører på servere).
4. DNS oversetter domenenavn (som `www.noroff.no`) til IP-adresser.
5. 404 betyr at ressursen du ba om ikke finnes (*Not Found*).

6. HTTPS er HTTP kryptert med TLS. Innholdet kan ikke leses eller endres underveis, og serverens identitet bekreftes med et sertifikat.
7. Et API er en avtalt måte for programmer å snakke sammen. Som en servitør som tar bestillingen din (forespørselen) til kjøkkenet (serveren) og kommer tilbake med maten (svaret), uten at du trenger å vite hvordan kjøkkenet fungerer.
8. Programvare som deler en fysisk maskins ressurser mellom flere virtuelle maskiner.
9. En komponent som stopper hele systemet hvis den feiler. Løses med redundans (flere eksemplarer).
10. Fordi alle handlinger i skyen – også klikk i portalen – er API-kall. Det du kan klikke, kan du også gjøre med kode.